

Dylan Cabahug-Almonte

☎ (484) 356-6487 | ✉ dcabahug@andrew.cmu.edu | 🔗 linkedin.com/in/dcalmonte | 📍 Pittsburgh, PA

Education

Carnegie Mellon University

August 2022 – December 2026

Bachelor of Science in Electrical and Computer Engineering | Minor in ML

Pittsburgh, PA

Relevant coursework: Data Structures & Algorithms, Distributed Systems, Embedded Systems, Database Systems, Deep Learning Systems, Generative AI, Parallel Computer Architecture

Skills

Programming: Python, C/C++, Go, Java, TypeScript, Bash, SQL

Frameworks & Libraries: React, PyTorch, LangChain, CUDA, NumPy, Java Spring Boot, REST APIs

Infrastructure: AWS (Lambda, S3), Linux, Docker, Cassandra, DuckDB, Jenkins, Git, CMake

Work Experience

Capital One

June 2026 – August 2026

Software Engineer Intern | Cyber

New York, NY

- Led a team of interns in designing system architecture, authoring entity-relationship diagrams and driving core design decisions for a full-stack KMS policy risk assessment tool on Capital One's GenAI Platform.
- Delivered and deployed the tool, scaling it to over 500 applications company-wide and automating identification and classification of KMS policy compliance gaps.
- Developed an agentic AI solution utilizing LLMs empowering developers to self-remediate KMS policy violations, eliminating manual security review overhead and streamlining compliance workflows.
- Identified 9 critical risk vulnerabilities across enterprise key policies and generated targeted remediation plans to address all of them, directly mitigating the organization's security exposure.

Carnegie Mellon University

Pittsburgh, PA

Student Research Assistant

August 2025 – Present

- Implemented LoRA-based fine-tuning pipelines for compact open-weight LLMs (LLaMA 3 8B) to encode domain-specific reasoning directly into model weights, reducing inference-time prompt complexity.
- Developed modular model routing infrastructure integrating contextual bandit algorithms to dynamically select between specialized models based on query characteristics and performance metrics.
- Engineered and benchmarked fine-tuning experiments comparing single vs. multi-adaptor architectures, evaluating tradeoffs in efficiency, task generalization, and degradation using quantitative LLM evaluation metrics.

Capital One

June 2025 – August 2025

Software Engineer Intern | EPTech

McLean, VA

- Engineered enhancements to an internal performance testing platform for Customer Relationships APIs (10K+ TPS, 5ms latency), optimizing tooling used to benchmark the reliability and efficiency of high-volume, mission-critical services.
- Integrated the testing system with 15+ microservices powering customer data for 185M+ users and 411M+ accounts, facilitating comprehensive API validation across all distributed services.
- Automated test execution and report generation pipelines using Python, Jenkins, and AWS (Lambda, S3), cutting manual overhead and shortening developer feedback cycles.

Ansys Inc.

January 2025 – April 2025

Software Developer Engineer Intern | Mechanical Team

Canonsburg, PA

- Led the migration of Visual Studio solutions to CMake, optimizing build processes and improving cross-platform compatibility for a large-scale C++ codebase, increasing development efficiency and maintainability.
- Developed and deployed comprehensive unit tests to improve test coverage and facilitate continuous integration, minimizing defects and strengthening software reliability across releases.

Projects

Query Execution Engine | *C++*

- Designed a row-based vectorized query execution engine supporting sequential scans, inserts, updates, deletes, and B+Tree index scans with dynamic key-size template dispatching.
- Architected hash join with Grace partitioning, nested loop/index joins, aggregation, external merge sort, window functions, and optimizer rules to convert sequential scans into index scans and nested loop joins into hash joins.

B+Tree Index | *C++*

- Implemented a concurrent B+Tree index supporting thread-safe search, insertion, and deletion with node splitting, merging, and key redistribution to maintain tree invariants.
- Designed a tombstone buffer on leaf pages for deferred deletions, maintaining tombstone ordering across split and coalesce operations as a simplified B ϵ -tree optimization.
- Applied optimistic latch crabbing with read/write page guards for concurrency control, enabling high-throughput parallel index operations without a global latch.

Text-to-Image Generation with Q-Former | *Python, PyTorch*

- Trained a Q-Former adapter bridging a frozen GPT-2 language model and a frozen Diffusion Transformer (DiT) for text-conditioned image generation on CIFAR-10, using classifier-free guidance.
- Constructed learnable query embeddings with multi-layer self-attention, cross-attention over GPT-2 hidden states, and per-layer projection to DiT conditioning vectors, allowing text prompts to drive diffusion sampling.

Raft Consensus Algorithm | *Go*

- Developed the Raft replicated state machine protocol with leader election, log replication, and commitment across a cluster of concurrent peers communicating via RPC.
- Engineered concurrent AppendEntries and RequestVote RPCs with election timeouts, heartbeats, and the election restriction to ensure consistent leader election under network partitions and failures.
- Handled log consistency with conflict detection, nextIndex backtracking, and majority-based commit advancement, ensuring linear state machine replication.

Mini Deep Learning Framework with Autograd and GPU Acceleration | *Python, C++, CUDA* October 2025

- Built a minimal deep learning framework from scratch implementing tensor operations, reverse-mode automatic differentiation, and dynamic computation graphs similar to PyTorch's Autograd engine.
- Implemented optimized GPU kernels using CUDA to accelerate matrix multiplication, convolution, and gradient propagation, achieving 20x speedup over CPU execution for standard benchmarks.
- Integrated modular components for model definition and training, enabling end-to-end execution of deep neural networks (MLPs and CNNs) with user-friendly APIs and performance profiling tools.

Fast Fourier Transform (FFT) | *Python, CUDA*

December 2025

- Implemented the Fast Fourier Transform (FFT) from first principles using the recursive Cooley–Tukey algorithm, reducing computational complexity from $O(N^2)$ to $O(N \log N)$.
- Validated correctness and numerical accuracy by comparing outputs against NumPy's backend implementation across multiple input sizes and signal types.
- Analyzed and benchmarked performance differences between naive DFT and FFT implementations to demonstrate scalability improvements for large signals.

Real Time Operating System | *C, ARM Assembly*

November 2024

- Developed a real-time kernel with functionalities for context switching, task scheduling, and mutex synchronization using ARM Cortex-M architecture.
- Implemented fixed-priority rate-monotonic scheduling (RMS) with schedulability checks using the Utilization Bound test to ensure real-time performance guarantees.
- Designed a thread control block (TCB) system to manage multi-threading, including priority-based scheduling.

Distributed Bitcoin Miner and Live Sequence Protocol (LSP) | *Go*

September 2024

- Accomplished a reliable communication protocol over UDP to manage packet loss, duplication, and out-of-order delivery. Features included sliding window protocol, exponential backoff for retransmissions, and checksum-based data integrity.
- Devised a client-server architecture where server distributed mining tasks to multiple miners. Server handled load balancing, fault tolerance, and task redistribution to optimize performance.
- Ensured robust fault tolerance with server-side task reassignment in case of miner failures and accomplished concurrency using Go's goroutines and channels to enable scalable, efficient shared computing.